

JOIN

Junções em Bancos de Dados

Um **JOIN** é um mecanismo utilizado para combinar informações armazenadas em tabelas diferentes. Em bancos de dados relacionais, os dados são separados para evitar redundância — e o JOIN é o que nos permite reunir tudo em uma única consulta.

Tabela Cliente

id_cliente	nome
1	João
2	Maria

Tabela Aluguel

id_aluguel	id_cliente	data
1	1	01/06
2	2	02/06

- ❓ Quem realizou cada aluguel? A tabela aluguel possui apenas o ID do cliente — precisamos buscar o nome na tabela cliente. É exatamente para isso que usamos JOIN.

Como funciona?

O Relacionamento Entre Tabelas

O JOIN funciona através de um relacionamento entre tabelas, normalmente estabelecido por meio de **Chave Primária (PK)** e **Chave Estrangeira (FK)**. O banco utiliza esses campos para descobrir quais registros pertencem uns aos outros.

Tabela CLIENTE

id_cliente (PK)

Chave Primária — identifica cada cliente de forma única na tabela.



Tabela ALUGUEL

id_cliente (FK)

Chave Estrangeira — referencia o cliente que realizou o aluguel.

- ❗ O banco de dados utiliza esses campos em comum para descobrir quais registros de uma tabela pertencem aos registros da outra.



INNER JOIN

O JOIN Mais Utilizado

O **INNER JOIN** retorna apenas os registros que possuem correspondência nas duas tabelas. Se um registro não tiver par na outra tabela, ele simplesmente não aparece no resultado.

Dados de Entrada

Cientes

id	nome
1	João
2	Maria
3	Pedro

Aluguéis

aluguel	cliente
1	1
2	2

Resultado do INNER JOIN

nome	aluguel
João	1
Maria	2

⚠ Pedro não aparece no resultado — ele nunca alugou nada, portanto não possui correspondência na tabela de aluguéis.

LEFT JOIN

Todos da Esquerda Aparecem

O **LEFT JOIN** retorna **todos** os registros da tabela da esquerda, mesmo que não exista correspondência na tabela da direita. Quando não há par, o resultado é preenchido com **NULL**.

Resultado do LEFT JOIN

nome	aluguel
João	1
Maria	2
Pedro	NULL

- ✔ Pedro aparece mesmo sem aluguel — com NULL no campo correspondente.

RIGHT JOIN – O Contrário

O **RIGHT JOIN** faz o oposto: retorna todos os registros da tabela da **direita**, mesmo que não existam na tabela da esquerda.

- ❏ Na prática, o RIGHT JOIN é pouco utilizado. A maioria dos desenvolvedores prefere inverter a ordem das tabelas e usar LEFT JOIN para manter consistência no código.

📋 **Analogia da Chamada:** Você faz a chamada da turma. Todos os alunos aparecem — mesmo os que não entregaram a atividade.

CROSS JOIN

Todas as Combinações Possíveis

O **CROSS JOIN** gera todas as combinações possíveis entre os registros de duas tabelas. Não há condição de relacionamento — cada linha de uma tabela é combinada com cada linha da outra.

Exemplo Prático

Clientes

João
Maria

Produtos

Camisa
Calça

Resultado

Cliente	Produto
João	Camisa
João	Calça
Maria	Camisa
Maria	Calça

A Fórmula do CROSS JOIN

Fórmula

Registros Tabela A ×
Registros Tabela B

Exemplo Real

5 clientes × 10 produtos =
50 combinações

⚠ Cuidado com tabelas grandes! O número de linhas cresce exponencialmente e pode sobrecarregar o banco de dados.

UNION

Empilhando Resultados

O **UNION** não junta tabelas lado a lado — ele **empilha resultados**. Essa é a principal diferença em relação ao JOIN.

JOIN	UNION
Combina colunas de tabelas diferentes em uma mesma linha.	Empilha registros de consultas diferentes em uma mesma coluna.
Exemplo: Cliente Cidade	Exemplo: João / Maria / Pedro

Exemplo de UNION

Funcionários

João
Maria

Clientes

Pedro
Carlos

Resultado Final

nome
João
Maria
Pedro
Carlos

 Regras: as consultas precisam ter o mesmo número de colunas e tipos compatíveis.

```
SELECT nome FROM cliente
UNION
SELECT nome FROM funcionario;
```

UNION vs UNION ALL

Duplicatas: Remover ou Manter?

A diferença entre **UNION** e **UNION ALL** está no tratamento de registros duplicados. Escolher o correto impacta tanto o resultado quanto a performance da consulta.

UNION

Remove registros duplicados automaticamente.

Resultado

João

Maria

Pedro

UNION ALL

Mantém todos os registros, incluindo duplicatas.

Resultado

João

Maria

Maria

Pedro

Qual é mais rápido?

UNION ALL é mais rápido porque não precisa verificar e eliminar duplicidades — ele simplesmente empilha tudo.

- ✔ Use **UNION ALL** sempre que tiver certeza de que não há duplicatas ou quando elas forem aceitáveis no resultado — você ganha performance.

GROUP BY

Agrupando Registros

O **GROUP BY** agrupa registros que possuem o mesmo valor em uma ou mais colunas. É amplamente utilizado para gerar relatórios e resumos de dados.

Exemplo Prático

Tabela Original

cidade
Brasília
Brasília
Goiânia
Goiânia
Goiânia

Resultado com GROUP BY

cidade	quantidade
Brasília	2
Goiânia	3

Por que usar GROUP BY?

Imagine um supermercado. Você quer saber:

- **Por funcionário**
Quantas vendas cada funcionário realizou
- **Por cidade**
Quanto clientes existem por cidade
- **Por categoria**
Quanto produtos existem por categoria
- **Por mês**
Quanto foi vendido por mês

⚠ Sem GROUP BY seria praticamente impossível gerar esses relatórios de forma eficiente.

Resumo

Todos os JOINS em um Só Lugar

Cada tipo de JOIN tem um propósito específico. Conhecer as diferenças entre eles é essencial para escrever consultas SQL eficientes e corretas.

1

INNER JOIN

Retorna apenas registros com correspondência nas duas tabelas. O mais utilizado no dia a dia.

2

LEFT JOIN

Retorna todos da esquerda + correspondências da direita. Sem par = NULL.

3

RIGHT JOIN

Retorna todos da direita + correspondências da esquerda. Pouco usado na prática.

4

CROSS JOIN

Gera todas as combinações possíveis. Resultado = A × B linhas.

5

UNION / UNION ALL

Empilha resultados de consultas. UNION remove duplicatas; UNION ALL mantém tudo.

6

GROUP BY

Agrupar registros com o mesmo valor para gerar relatórios e contagens.

SQL

Domine as Consultas

Compreender JOINS, UNIONS e GROUP BY é a base para trabalhar com bancos de dados relacionais de forma profissional. Com esses recursos, você consegue responder praticamente qualquer pergunta sobre seus dados.



Pratique

Crie tabelas de exemplo e teste cada tipo de JOIN para fixar o aprendizado.



Compare

Observe as diferenças nos resultados ao trocar INNER por LEFT JOIN.



Aplique

Use GROUP BY para gerar relatórios reais com dados agrupados e contados.



Com domínio de JOINS e GROUP BY, você está pronto para construir consultas SQL poderosas e gerar insights valiosos a partir de qualquer banco de dados relacional.

